

# There Is Always a Way Out!

## Destruction-Resistant Key Management

Yaqing Song<sup>1</sup>, Yuan Zhang<sup>1\*</sup>, Shiyu Li<sup>2</sup>, Weijia Li<sup>1</sup>,  
Zeqi Lai<sup>3</sup>, and Qiang Tang<sup>4</sup>

<sup>1</sup> University of Electronic Science and Technology of China, Chengdu, China  
YaqingS@163.com, zhangyuan@uestc.edu.cn, tokio\_0@163.com

<sup>2</sup> The University of Virginia, tkp2ra@virginia.edu

<sup>3</sup> Tsinghua University, Beijing, China zeqilai@tsinghua.edu.cn

<sup>4</sup> The University of Sydney, Sydney, Australia qiang.tang@sydney.edu.au

**Abstract.** A central advantage of deploying cryptosystems is that the security of large high-sensitive data sets can be reduced to the security of a very small key, i.e., a master key. The most popular way to manage the master key is to use  $(t, n)$ –threshold secret sharing schemes: a user splits and distributes her/his key among  $n$  key servers, and can recover the key with the aid of any  $t$  of them. However, it is vulnerable to device destruction: if all key servers and user’s devices break down, the key will be permanently lost. We propose a Destruction-Resistant Key Management scheme, dubbed DRKM, which ensures the key availability even if destruction occurs. In DRKM, a user utilizes her/his  $n^*$  personal identification factors (PIFs) to derive a cryptographic key but can retrieve the key using any  $t^*$  of the  $n^*$  PIFs. As most PIFs can be retrieved by the user *per se* without requiring *stateful* devices, destruction resistance is achieved. With the integration of a  $(t, n)$ –threshold secret sharing scheme, DRKM achieves *portable* key access by invoking any  $t$  key servers before destruction occurs. We prove the DRKM’s security, implement its prototype, and conduct a comprehensive performance evaluation to demonstrate its high efficiency.

**Keywords:** Destruction resistance · Key management · Reconstructable PIFs

## 1 Introduction

Secure and efficient key management schemes are cornerstones of any cryptosystem, which should satisfy the desired requirements of availability (users can always correctly recover their keys) and portability (users can access their keys from multiple devices). To this end, a user always stores her/his cryptographic keys in a repository [2,3,4,49,77]. Generally, the repository can be either instantiated by deploying a local device [77] or subscribing to key access services from a dedicated service provider [4]. Such a repository always refers to a key server in

---

\* Corresponding author

literature [28,44]. This paradigm has been widely utilized in commercial systems, e.g., Microsoft Azure [3] and Google Cloud Platform [2].

Despite the advantage of deploying a key server, a critical issue—vulnerability of the system against device destruction—arises naturally: as uncontrollable and unpredictable threats towards the key server always exist in reality, the user has to bear the risk that her/his keys would be permanently lost if the key server is destroyed. We stress that device destruction is not just a theoretical concern, and recent incidents have shown that it would happen with various manifestations which typically consist of hardware destruction and software unavailability [59,57,62]. Notably, a private key stored in a hard drive was permanently unavailable to its owner due to a hard drive breakdown, which directly caused that 7500 bitcoins (which are worth more than \$280 million today) could never be used by the owner [59]. In addition, once the key server storing users' cryptographic keys is hacked by ransomware attacks, all cryptographic services could be paralysed [57,62]. As such, remaining availability in case of device destruction has become a primary requirement for cryptographic key management systems.

The most popular method to manage cryptographic keys is the threshold secret sharing scheme [65] (as well as its variants [53]), where  $n$  key servers are deployed, and each of them maintains a share of the key such that the key can be recovered with any  $t$  shares. Such a key management scheme provides a strong guarantee in terms of security and reliability: even if an adversary compromises  $t - 1$  key servers, it cannot get any information about the key; the destruction of any  $n - t$  key servers cannot hamper the key recovery. Due to the theoretically desirable properties and practical natures, threshold secret sharing schemes still serve as a key component for lots of high-sensitive systems (e.g., vault systems<sup>5</sup>) in the current age, even though the pioneering work was proposed by Shamir [65] more than 40 years ago. However, the fundamental issue of remaining at least  $t$  key servers available under any circumstance still exists. In reality, misfortunes causing simultaneous destruction of all key servers could still happen in any system, no matter what high degree of reliable measures would be taken. For instance, Amazon Web Services (AWS) suffered a major outage [7] due to misoperations, where all servers in the Amazon Simple Storage Service (S3) subsystems broke down, and many popular websites, e.g., Netflix and Slack, were affected [9]. Severe natural disasters would also directly destroy local servers and make them permanently unavailable to their users. A notable example is that the eruption of the Tonga volcano in 2022 destroyed critical information infrastructure almost all over the country.<sup>6</sup>

A natural way to mitigate this problem is to employ additional servers providing backup services: if some key servers break down, backup servers can continue to handle users' requests.<sup>7</sup> Nevertheless, this remedy cannot be applied for

<sup>5</sup> <https://developer.hashicorp.com/vault>

<sup>6</sup> <https://reliefweb.int/disaster/vo-2022-000005-ton>

<sup>7</sup> To resist the destruction caused by natural disasters, the backup servers can be deployed around the world. However, this approach may violate data protection regulations in several countries [51] and would be expensive to deploy in practice.

key management, since multiple backups of keys increase the danger of security breaches. This motivates us to consider the following question:

*Can we have a key management scheme that ensures key availability even if all repositories (including users' devices and key servers) are destroyed?*

The key observation behind our work is that destruction-resistant key availability can only be achieved by a key generation mechanism that enables the user to recover the key as needed without requiring any *stateful*<sup>8</sup> device. With the observation, we introduce two new concepts: reconstructable secret and unreconstructable secret, depending on whether a stateful device is necessary for recovering the secret. We then propose a practical key derivation mechanism to generate reconstructable secrets, where the key idea is to generate the reconstructable secret using users' personal identification factors (PIFs).<sup>9</sup> Specifically, we categorize PIFs into three types: device-dependent ones, device-independent ones, and storage-independent ones, where processing device-dependent/device-independent PIFs (for cryptography purposes), such as biometric characteristics [70,24,74], requires a stateful/stateless device, and processing storage-independent PIFs, e.g., passwords, does not even require any storage. A systematic analysis is provided in Section 2. We also notice that reconstructable secrets can be directly derived from device/storage-independent PIFs. With reconstructable secrets, it seems that a destruction-resistant key management scheme can be trivially constructed: a user constructs a master key from multiple reconstructable secrets and further utilizes it to derive other cryptographic keys. However, the above scheme is also confronted with the following issues.

Regarding functionality, the key recovery depends on a strong assumption that the user needs to keep all device-independent PIFs available under any circumstance. As a counterexample, if the user utilized a fingerprint to generate the master key, when large-scale disasters occur, the user's finger may be injured, and consequently the user cannot recover the master key until the finger heals. Regarding convenience, the portability is also lost, since "device-independent" is not equivalent to "portable" (even if a PIF is device-independent, the user may not retrieve it anytime and anywhere). For instance, if a user has a camera capable of collecting irises, she/he can derive a secret from the iris. Subsequently, the user can only recover the secret when such a specific-purpose device (that may not be the same as the previous one but has the same functionalities) is equipped. Such a secret fails to achieve portability, and migrating it may cause new issues in terms of security and efficiency.

The above limitations further motivate us to consider the following question:

---

<sup>8</sup> *Stateful* means that the device stores some secret information related to the user.

<sup>9</sup> We utilize the terminology of "PIFs" here to distinguish from authentication factors (AFs). A PIF is the factor that uniquely identifies a user while an AF is considered as a special PIF that can be utilized to construct secure and usable authentication schemes. Some PIFs cannot serve as AFs, e.g., DNA is a PIF but cannot be used to construct usable authentication schemes (due to its inconvenience and high costs).

*Can we have a destruction-resistant key management scheme that enables key recovery from a subset of original PIFs while achieving portability before destruction occurs?*

We stress that the conventional threshold secret sharing [65] and its distributed variants [53] fail to achieve the key recovery from a subset of original PIFs, since they essentially share the same paradigm: first determine the secret and then split it into multiple shares; any threshold number of shares can reconstruct the secret. However, in a destruction-resistant key management scheme introduced before, the “shares” (i.e., the reconstructable secrets) are pre-determined by PIFs, and the master key is derived from them. To achieve the key recovery from a subset of original PIFs, a threshold key derivation mechanism should be designed, such that the master key can be “derived” from all pre-determined “shares” but can be recovered with only the threshold number of them. By comparison, the “shares” are determined by PIFs rather than the master key as in conventional threshold secret sharing.

To achieve portability, the user can derive  $n$  different reconstructable secrets from different PIFs, employ  $n$  key servers, and let each key server maintain one secret. By doing so, the user can access the master key with the aid of key servers in a portable way<sup>10</sup> before the destruction occurs. Whereas, such an approach is vulnerable to trawling attacks [73]. Specifically, a PIF is not only used for generating the master key in one system but also used in other systems for other cryptographic purposes, e.g., secure authentication. In the above approach, the key servers can compromise enough information about the user’s PIFs from the reconstructable secrets, which enables adversarial key servers to impersonate the user to access other services where the same PIFs are used for authentication.

*To the best of our knowledge, we still lack a destruction-resistant key management scheme that enables key recovery from a subset of original PIFs after destruction occurs while achieving portability before destruction occurs.*

## 1.1 Our contributions

In this paper, we propose a Destruction-Resistant Key Management scheme, dubbed DRKM, which goes one step beyond existing schemes [65,53]. Specifically, our contributions are summarized as follows.

**New concepts about PIFs and secrets.** We first propose three new concepts about personal identification factors (PIFs)—storage-independent PIFs, device-independent PIFs, and device-dependent PIFs—based on whether a PIF can be retrieved by the user *per se* without requiring any storage or a *stateful* device. Based on these new concepts, we further introduce two concepts about secrets—reconstructable secrets and un-reconstructable secrets. We point out that reconstructable secrets can be directly derived from storage/device-independent PIFs. We also present a series of methods to derive reconstructable secrets from device-dependent PIFs in tandem with storage/device-independent PIFs under certain conditions.

<sup>10</sup> The user can authenticate herself/himself with portable PIFs.

**Construction for destruction-resistant key management.** We propose DRKM, a destruction-resistant and portable key management scheme. To achieve destruction resistance, DRKM utilizes a threshold key derivation mechanism to enable a user to derive a master key from  $n^*$  PIFs (which include storage/device-independent ones and might include device-dependent ones) during the setup phase and to recover the master key using any  $t^*$  of the  $n^*$  PIFs after destruction occurs. To achieve portability, DRKM adopts a multi-server-aided paradigm and utilizes a conventional  $(t, n)$ -threshold secret sharing scheme ( $t$  and  $n$  are independent of  $t^*$  and  $n^*$ ) to distribute the master key among  $n$  key servers. As long as any  $t$  of  $n$  key servers are available, users can access the master key in a portable way (i.e., they do not maintain any secret in local devices). DRKM is compatible with existing backup systems and can be directly extended to a destruction-resistant cryptosystem (DRC) in tandem with any commercial cloud storage service, such as Google Drive.<sup>11</sup>

**Formal security proofs and prototype implementation.** We provide formal security definitions of DRKM and prove its security. We prove that an adversary, who compromises  $t - 1$  key servers and  $t^* - 1$  PIFs, cannot get any extra information about the master key derived from PIFs. We implement a DRKM prototype and conduct a comprehensive performance evaluation.

We demonstrate the viability of DRKM for two existing applications that can benefit from the desirable property of destruction resistance in Section 6.3. One of two applications extends to DRCs. We show how DRKM supports these applications without changing the current system architecture. Since destruction resistance is a fundamental requirement of any cryptosystem, we believe that DRKM has further useful applications.

## 1.2 Technical overview

The core of achieving destruction resistance is to be free from the reliance on stateful devices. In Section 2, we divide PIFs into three categories: storage-independent PIFs, device-independent PIFs, and device-dependent PIFs. A user can directly derive reconstructable secrets from storage/device-independent PIFs, and these secrets are independent of any stateful devices. We also present a series of methods to derive reconstructable secrets from device-dependent PIFs in conjunction with storage/device-independent PIFs.

With the above methods, a user first derives  $n^*$  reconstructable secrets from PIFs and then aggregates these secrets to obtain a master key. The challenge in designing DRKM is to achieve threshold retrieval for the aggregated master key, i.e., a master key is aggregated from  $n^*$  pre-determined secrets and can be retrieved from any  $t^*$  of them. To address the challenge, we utilize a threshold key derivation mechanism. Specifically, the master key is derived from the  $n^*$  secrets. The user constructs a  $n^*$ -degree polynomial  $p(x)$  using the  $n^*$  secrets as its roots, and the coefficients of  $p(x)$  of degree  $n^* - 1$  down to  $t^*$  are published as the auxiliary information  $aux$ . With  $t^*$  secrets and  $aux$ , the degree of the

<sup>11</sup> <https://www.google.com/intl/en-GB/drive/>

polynomial  $p(x)$  can be reduced from  $n^*$  to  $t^* - 1$ . The user can compute the coefficients of  $p(x)$  from degree  $t^* - 1$  down to 0, then find all roots of  $p(x)$ , and then recover the master from these roots, i.e., the secrets.

We also integrate an aggregation-then-split mechanism into DRKM to achieve portable key access in normal times against trawling attacks. The master key derived from the  $n^*$  reconstructable secrets is further split into  $n$  shares using a conventional  $(t, n)$ -threshold secret sharing scheme, and each key server maintains a share. Adversarial key servers cannot compromise any information about the reconstructable secrets from the secret shares. This yields the final DRKM: before the destruction occurs, the user can recover the master key with the aid of key servers in a portable way; once the destruction occurs (i.e., the key servers and the user's devices are destroyed), the user can retrieve any  $t^*$  reconstructable secrets using all available PIFs at that time and recover the master key from them. In the extreme case where all devices, key servers, and even *aux* is unavailable, the user can also recover the master key from  $n^*$  original PIFs. A destruction-resistant cryptosystem (DRC) can be developed by directly integrating DRKM and a full-fledged backup system.

### 1.3 Comparison with concurrent work

A very recent work concurrent to DRKM (i.e., threshold multi-factor key derivation function, short for TMFKDF) proposed by Nair et al. [55] could be a partial solution to construct a destruction-resistant key management scheme: a user first randomly chooses a master key, splits it into multiple shares using a conventional threshold secret sharing scheme, and encrypts each share under a PIF.

Essentially, TMFKDF inherits the threshold property of the conventional threshold secret sharing scheme, where the master key is randomly chosen by the user rather than determined by PIFs. Therefore, some metadata, e.g., the encrypted shares, is inherently needed for key recovery, and dedicated storage for metadata is always required. By comparison, DRKM is completely orthogonal to the conventional sample-share-and-reconstruct idea as in TMFKDF. With DRKM, the user can also recover the master key from  $n^*$  reconstructable secrets even if any storage is unavailable. In DRKM, it seems that some metadata is also required in some cases. For instance, if the user derives reconstructable secrets from biometric characteristics [70, 74], some metadata, e.g., error correcting code [32], is required to ensure the consistency of secrets in different derivations. However, we stress that such metadata in DRKM is totally different from ciphertext in TMFKDF due to the following reasons.

The metadata in DRKM can be shared among different systems where PIFs are utilized for other purposes, e.g., user authentication. As such, dedicated storage is not required, and the user can retrieve the metadata from other systems on demand. However, for TMFKDF, dedicated storage for ciphertext cannot be shared among other systems. Furthermore, it is promising to be free from metadata in DRKM by utilizing new PIFs, where deriving secrets from them does not require any storage. A detailed comparison is provided in Section 6.1.

**Roadmap.** The remainder of this paper is organized as follows. We introduce the concepts of storage/device-independent PIFs in Section 2. We propose DRKM in Section 3 and give the formal security proof in Section 4. In Section 5, we detail the implementation and evaluate the performance of DRKM. We provide comparisons and applications in Section 6 and review the related works in Section 7. Finally, we draw the conclusion and outlook for future research directions in Section 8.

## 2 PIFs and reconstructable secrets

### 2.1 Preliminaries

**Notation.**  $\ell$  denotes the security parameter, and  $\vec{A}$  denotes a set  $\{A_1, \dots, A_n\}$ .  $[1, n]$  denotes the set  $\{1, 2, \dots, n\}$ .  $a \xleftarrow{\$} A$  denotes that  $a$  is uniformly chosen from  $A$ .  $a||b$  denotes the concatenation of two strings  $a, b$ .  $Z_p$  denotes the set  $\{0, 1, \dots, p-1\}$ , where  $p$  is a prime.

**Min-entropy** [66]. For adversaries, an attack strategy is guessing random values used in cryptosystems (e.g., secret keys). The probability of an adversary guessing out the key is determined by the entropy of the key. In information theory, the entropy of a random variable is the level of uncertainty inherent in the random variable's possible outcomes.

Shannon [66] first introduced the concept of information entropy.  $A = \{a_1, a_2, \dots, a_n\}$  is a discrete and finite set. Assume that  $X$  is a random variable where the value domain is  $A$  and the probability of choosing  $a_i$  from  $A$  is  $p_i$ , i.e.,  $\Pr[X = a_i] = p_i$ . The information entropy of  $X$  is

$$H(X) = - \sum_{i=1}^n p_i \log p_i.$$

The predictability of the random variable  $X$  is  $\max\{p_i\}_{i \in [1, n]}$  that corresponds to the min-entropy of  $X$  is

$$H_{\infty}(X) = -\log(\max\{p_i\}_{i \in [1, n]}).$$

**Key derivation function.** A key derivation function can be utilized to derive a cryptographic key from a random source. We follow the security definition of KDF proposed in [46], as shown in Definition 1.

**Definition 1.** A key derivation function (KDF) is secure with respect to an  $m$ -min-entropy source of key material  $\Sigma$  if no adversary  $\mathcal{A}$  can distinguish the key generated from  $\Sigma$  and a random string of the same length with probability better  $1/2 + 2^{-m}$ .

## 2.2 Definitions of PIFs

We analyze popular PIFs and give a brief introduction to them in Section 7.1, referring to “something the user knows”, “someone the user is”, and “something the user has” [22]. Intuitively, a PIF can uniquely identify a user, and thereby each PIF indicates a sole secret utilized to distinguish different PIFs. We observe that the utilization of some PIFs has to depend on stateful hardware devices that maintain the necessary state information, e.g., hardware tokens. Additionally, some auxiliary information is required, which would include the directions for use and necessary descriptions. According to the definition of the factor in [55], we formally define a general PIF and its min-entropy as follows.

**Definition 2.** *A PIF is a triple-valued probability distribution  $(U, A, \text{State})$  which is generated by an efficient probabilistic algorithm.*

**Definition 3.** *A PIF has  $m$ -min-entropy if for  $\forall(\mu, \alpha, \text{sta})$  which satisfies its probability distribution,  $\Pr[U = \mu | A = \alpha, \text{State} = \text{sta}] \leq 2^{-m}$ .*

For a PIF instance  $(\mu, \alpha, \text{sta})$ ,  $\mu$  is a private value, and  $\alpha$  is public;  $\text{sta}$  would be public or private, depending on specific PIFs. For clarity, we take passwords, SIM cards [50], and biometric characteristics as examples. For a password,  $\mu$  is the password,  $\alpha$  would be some public information, e.g., the strength requirements, and  $\text{sta} = \perp$ , since passwords are human-memorable and independent of any extra stateful information. For a SIM card,  $\mu$  is the secret key fused in it,  $\alpha$  includes some public parameters and authentication protocols that are utilized, and  $\text{sta}$  represents some private *stateful* information (which can be extracted from the SIM card). For a biometric characteristic,  $\mu$  is the encoded biometric feature,  $\alpha$  would contain the recognition and encoding methods,  $\text{sta}$  represents some public information (which can be extracted from dedicated equipment, e.g., one used to collect biometric features). With the previous analyses, we can heuristically divide the above PIFs into three categories as follows.

*Storage-independent PIFs.* If a PIF  $(U, A, \text{State})$  satisfies  $\text{State} = \perp$ , then it is storage-independent. In reality, passwords and PIN codes are storage-independent and can be reconstructed from users’ memory and are inherently independent of any personal or public storage.

*Device-independent PIFs.* If a PIF  $(U, A, \text{State})$  satisfies that  $\text{State}$  is public, then it is device-independent. We observe that all biometric characteristics are device-independent, as they are determined by a user per se. However, they may not totally independent of storage, since some public information, e.g., error correcting code [32], is needed when utilizing them. In cryptographic applications, how to make biometric characteristics free from dedicated storage is a fascinating open problem.

*Device-dependent PIFs.* If a PIF  $(U, A, \text{State})$  satisfies that  $\text{State}$  is private, then it is device-dependent. SIM cards, hardware token are device-dependent PIFs, which rely on specific devices. For device-dependent PIFs (e.g., SIM cards, hardware tokens), once a stateful device is destroyed, the corresponding PIF cannot be recovered by the user per se.



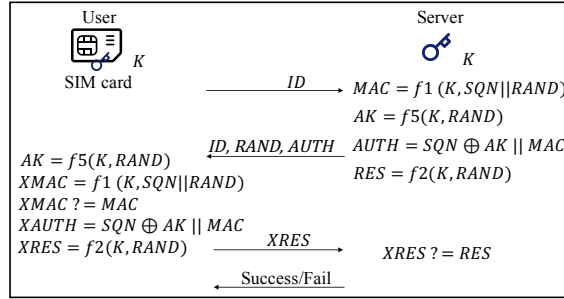


Fig. 1: User authentication based on SIM cards.

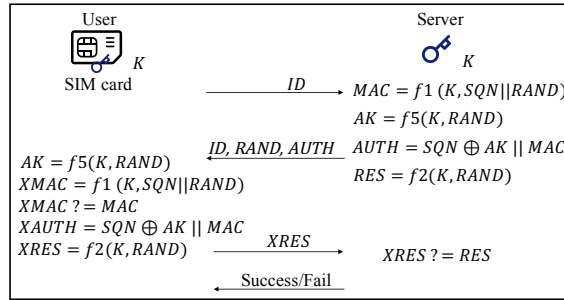


Fig. 2: User authentication based on RSA SecurIDs.

Obviously, storage-independent PIFs are also device-independent but device-independent ones need public information for reconstruction and are dependent on public storage.

### 2.3 Reconstructable secrets

We first consider what it means to be “reconstructable”. Informally, a value is reconstructable if it is available without a specific stateful device and can be accessed anytime and anywhere. Reconstructable values generalize the notion of storage/device-independent PIFs. We can trivially extend reconstructable values to *reconstructable secrets* by further requiring the value to be kept secretly. Similarly, a secret derived from an un-reconstructable value is un-reconstructable.

In reality, we can utilize storage/device-independent PIFs to serve as reconstructable values to derive reconstructable secrets. For instance, fuzzy extractor algorithms [32] can be utilized to extract a secret from a feature template of biometric characteristics. The secret is reconstructable, since fuzzy extraction algorithms ensure that the same secret can be extracted from similar but not identical feature templates. Directly (only) using passwords and PINs as reconstructable secrets cannot achieve a reasonable security guarantee due to their

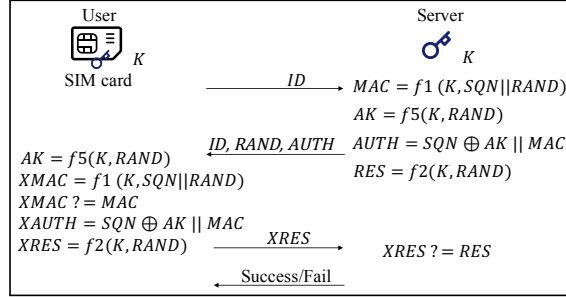


Fig. 3: User authentication based on U2F tokens.

inherent limitations. To mitigate this problem, we can integrate multiple PIFs to derive a high min-entropy reconstructable secret.

## 2.4 Conditionally reconstructable secrets

It seems impractical to derive a reconstructable secret from a device-dependent PIF, since a secret derived from a device-dependent PIF cannot be recovered by the user if the device is destroyed. However, we observe that a *conditional* reconstructable secret can be generated by a “hybrid model”, i.e., we can derive a reconstructable secret from a special class of device-dependent PIFs in tandem with some storage/device-independent PIF(s) under a specific assumption.

For the device-dependent PIFs introduced in Section 7.1, i.e., SIM cards, hardware tokens, and SGX, they essentially share the same paradigm, where a secret generated by the manufacturer is fused in the device. The user can only utilize the secret to compute authentication credentials but cannot extract it. We observe that if the underlying authentication scheme is based on the symmetric-key cryptographic primitives (e.g., MAC), the server will store the same secret that is fused in the device after registration. In this case, once the device is destroyed, the server (or the device’s manufacturer) still stores the secret. As such, the secret can be recovered with the aid of the server or the manufacturer even if device destruction occurs.

In the following, we discuss how to derive a conditionally reconstructable secret from each of the device-dependent PIFs introduced in Section 7.1.

*SIM card.* As shown in Figure 1, a user can utilize a SIM card to compute a MAC on a storage/device-independent PIF and set the MAC as the conditionally reconstructable secret derived from the SIM card. Since MAC is existentially unforgeable, and the PIF is secretly maintained by the user, the MAC-based secret is only known to the user. In addition, as the secret fused in the SIM card is also maintained by the cellular communication service provider, the user can recover the MAC-based secret with the aid of the provider.

*Hardware token.* For HMAC-based hardware tokens (as shown in Figure 2), the user can derive a conditionally reconstructable secret in the same way as that

from the SIM card. However, ECDSA-based hardware tokens (as shown in Figure 3) cannot be utilized to derive a reconstructable secret, since the manufacturer does not maintain the secret stored in the user’s hardware token. However, we notice that for some existing hardware tokens, this can be achieved by utilizing a well-known attack, i.e., the backdoor attack released by Snowden [6], where this approach actually does not need the assistance from the manufacturer [31].

*Intel SGX.* The Root Provisioning Key (PRK) fused in Intel SGX is shared by a user and Intel. Intel SGX architecture provides the *EGETKEY* instruction to derive a key from the RPK [29,13]. The user initializes an enclave for a storage/device-independent PIF and invokes *EGETKEY* for the enclave. The key output by the *EGETKEY* is the reconstructable secret derived from the Intel SGX. When reconstructing the secret, the user only needs to establish an enclave for the previous storage/device-independent PIF and get the secret by calling the *EGETKEY* instruction.

*Limitations of hybrid model.* Regarding security, the user has to fully trust the servers or the manufacturers. Malicious servers and manufacturers may abuse the user’s secret. Regarding reliability, the reconstructable secrets derived by the hybrid method are conditionally reconstructable. The servers or manufacturers have another mechanism to authenticate the user when the user’s devices are destroyed, e.g., real-name systems. The reconstructability depends on the reliability of the servers or the manufacturers. Once the servers and manufacturers are destroyed, the user cannot recover the reconstructable secrets.

*It is worth stressing that the above limitations are not contradictory to our primary motivation of resistance against device destruction* due to the following reasons. First, the manufacturers and cellular communication service providers play an important role in constructing critical infrastructures, and thereby they would take lots of measures to ensure the reliability and security of their devices. Second, if the manufacturers and cellular communication service providers misbehave, it would cause a huge loss. As such, both the manufacturers and cellular communication service providers bear rigorous accountability from governments in reality. By comparison, the measures taken by application service providers and users to improve the reliability and security of key servers and devices are always weak, and the accountability is somewhat trivial.

### 3 The proposed DRKM

#### 3.1 Definition of DRKM

DRKM consists of three entities: a user  $\mathcal{U}$ , a set of key servers  $\overrightarrow{\mathcal{KS}} = \{\mathcal{KS}_1, \mathcal{KS}_2, \dots, \mathcal{KS}_n\}$ , and a backup system. There are four algorithms in DRKM, **Setup**, **Managing**, **Access**, and **Recovery**.

- $PP \leftarrow \text{Setup}(\ell)$ .

On input the security parameter  $\ell$ , it returns public parameters  $PP$ , where two thresholds, i.e.,  $(t, n)$  and  $(t^*, n^*)$ , are included.  $(t, n)$  is independent of  $(t^*, n^*)$ .

- $\{S, s_1, \dots, s_n, aux\} \leftarrow \mathbf{Managing}(PIF_1, \dots, PIF_{n^*})$ .  
On input  $n^*$  PIFs  $\{PIF_1, \dots, PIF_{n^*}\}$ , this algorithm returns a master key  $S$ ,  $n$  shares  $\{s_1, \dots, s_n\}$  of  $S$ , and auxiliary information  $aux$ .  $\mathcal{U}$  generates a master key  $S$  using  $\{PIF_1, \dots, PIF_{n^*}\}$ , sends the secret share  $s_i$  to  $\mathcal{KS}_i$ .  $\mathcal{U}$  stores  $aux$  with a backup system.
- $S \leftarrow \mathbf{Access}(s_{i_1}, \dots, s_{i_t}, aux)$ .  
On input any  $t$  of  $n$  shares  $\{s_{i_1}, \dots, s_{i_t}\}$ , it returns  $S$ .  $\mathcal{U}$  gets  $\{s_{i_1}, \dots, s_{i_t}\}$  from  $t$  key servers and  $aux$  from the backup system and can access  $S$ .
- $S \leftarrow \mathbf{Recovery}(PIF_{i_1}, \dots, PIF_{i_{t^*}}, aux)$ .  
On input  $t^*$  PIFs  $\{PIF_{i_1}, \dots, PIF_{i_{t^*}}\}$ , and  $aux$ , it returns the master key  $S$ . If the number of available key servers is less than  $t$ ,  $\mathcal{U}$  utilizes available PIFs  $\{PIF_1, \dots, PIF_{t^*}\}$  to recover  $S$ .

### 3.2 Functionality of DRKM

The primary functionality of DRKM is to ensure that users can recover their keys when both key servers and devices are unavailable. We refine three cases and discuss the functionality that should be satisfied case by case as follows.

- *Normal times.* In normal times, both dedicated storage for sensitive information and general storage for non-sensitive information are available to users, i.e., any  $t$  key servers and the backup system are available. DRKM should achieve portability in normal times, i.e., enable users to access their master keys without maintaining any secret locally.
- *Partial destruction.* Partial destruction means that dedicated storage for sensitive information is destroyed but general storage for non-sensitive information is available to users. It indicates that all key servers and devices are destroyed but the backup system and any  $t^*$  of  $n^*$  PIFs are available. In this case, DRKM should enable users to recover their master keys.
- *Full destruction.* In the full destruction case, neither dedicated storage for sensitive information nor general storage for non-sensitive information is available to users, i.e., all key servers and devices are destroyed, and even the backup system is unavailable to users. Under this circumstance, DRKM should ensure the key recovery if all  $n^*$  PIFs are available.

### 3.3 Security of DRKM

The security goals of DRKM are as follows. 1) The masker key derived from PIFs is a cryptographic key. 2) An adversary who compromises any  $t - 1$  shares and any  $t^* - 1$  PIFs would not get any additional information about the master key. The security property of  $(t, t^*, n, n^*)$ -DRKM is captured by Definition 4.

**Definition 4.** *DRKM is secure if the master key output by the **Managing** algorithm is computationally indistinguishable from a uniformly-chosen key from  $\{0, 1\}^\ell$  assuming any  $t - 1$  of  $n$  shares and any  $t^* - 1$  of  $n^*$  PIFs are leaked, where  $\ell$  is the security parameter.*

### 3.4 Construction of DRKM

We instantiate the backup system using a cloud server  $\mathcal{CS}$ . A user  $\mathcal{U}$ , a set of key servers  $\vec{\mathcal{KS}} = \{\mathcal{KS}_1, \dots, \mathcal{KS}_n\}$ , and a cloud server  $\mathcal{CS}$  are involved in DRKM.

**Setup.** With the security parameter  $\ell$ , the parameters  $\{p, \mathbf{F}, \mathbf{H}, t^*, t, n^*, n, \}$  are determined and published, where  $p$  is a prime with  $\log p \leq \ell$ ,  $\mathbf{F}$  is a secure key derivation function<sup>12</sup>,  $\mathbf{H} : \{0, 1\}^* \rightarrow Z_p$  is a secure hash function,  $t^*$  and  $t$  are two thresholds,  $n^*$  is the number of PIFs, and  $n$  is the number of key servers. We assume that  $\mathcal{U}$  derives  $n^*$  reconstructable secrets from  $n^*$  PIF instances  $\vec{PIF} = \{PIF_1, \dots, PIF_{n^*}\}$  respectively, where these PIFs' distributions are independent. We require that the minimum among the min-entropy values of all PIFs used for key management is not less than  $\lambda$ .<sup>13</sup>

**Managing.**  $\mathcal{U}$  utilizes the PIFs to manage  $\vec{K}$  as follows.

- For  $PIF_i \in \vec{PIF}$  and  $i = 1, 2, \dots, n^*$ , if  $PIF_i$  is storage/device-independent,  $\mathcal{U}$  can compute a reconstructable secrets from it, and if  $PIF_i$  is device-dependent,  $\mathcal{U}$  can compute a conditionally reconstructable secrets from it. The reconstructable secret is  $seed_i = \mathbf{F}(PIF_i)$ .
- $\mathcal{U}$  computes  $s_i^* = \mathbf{H}(seed_i)$ .
- $\mathcal{U}$  generates a polynomial  $p(x)$  over  $Z_p$

$$p(x) = \prod_{i=1}^{n^*} (x - s_i^*),$$

such that  $p(s_i^*) = 0$  for  $i = 1, 2, \dots, n^*$ .

- $\mathcal{U}$  outputs the coefficients of  $p(x)$  of degree  $n^* - 1$  down to  $t^*$  as the auxiliary information  $aux$ , i.e.,

$$aux = \left\{ -\sum_{i=1}^{n^*} s_i^* \mod p, \sum_{\substack{A \subseteq [1, n^*] \\ |A|=2}} (\prod_{i \in A} s_i^*) \mod p, \right. \\ \left. -\sum_{\substack{A \subseteq [1, n^*] \\ |A|=3}} (\prod_{i \in A} s_i^*) \mod p, \dots, (-1)^{n^*-t^*} \cdot \sum_{\substack{A \subseteq [1, n^*] \\ |A|=n^*-t^*}} (\prod_{i \in A} s_i^*) \mod p \right\}.$$

- $\mathcal{U}$  computes  $S = \mathbf{H}(s_1^* || s_2^* || \dots || s_{n^*}^*)$  as the master key.
- $\mathcal{U}$  uniformly chooses  $a_1, \dots, a_{t-1} \xleftarrow{\$} Z_p$  and generates a polynomial  $f(x) = S + a_1x + \dots + a_{t-1}x^{t-1}$  over  $Z_p$  with degree at most  $t - 1$ .
- $\mathcal{U}$  computes  $s_i = f(i)$  for  $i = 1, 2, \dots, n$ .
- $\mathcal{U}$  establishes a secure channel with  $\mathcal{KS}_i$  and sends  $s_i$  to  $\mathcal{KS}_i$ .  $\mathcal{U}$  outsources  $aux$  to  $\mathcal{CS}$ .

<sup>12</sup> We do not specify the length of the input and output of  $\mathbf{F}$ , since the length depends on which PIF is utilized.

<sup>13</sup> In reality, users can derive a secret from multiple PIFs to achieve a higher security level, i.e., the number of PIFs can be different from that of reconstructable secrets. For the sake of simplicity, we assume that they are equal and only describe how to derive a secret from a single PIF.

- $\mathcal{KS}_i$  securely stores  $s_i$ .

**Access.**  $\mathcal{U}$  accesses  $S$  with the aid of  $\overrightarrow{\mathcal{KS}}$  as follows.

- $\mathcal{U}$  establishes a secure channel with  $\mathcal{KS}_i$ , and  $\mathcal{KS}_i$  sends  $s_i$  to  $\mathcal{U}$  via the secure channel. This can be achieved by utilizing an authentication scheme based on other storage/device-independent PIFs.
- Upon receiving  $t$  secret shares (denoted by  $\{s_1, \dots, s_t\}$ ),  $\mathcal{U}$  computes  $S$  as

$$S = \sum_{i=1}^t w_i s_i, \text{ where } w_i = \prod_{\substack{1 \leq j \leq t \\ j \neq i}} \frac{j}{j-i}.$$

**Recovery.** When the number of available key servers is less than  $t$ ,  $\mathcal{U}$  recovers  $S$  with available PIFs as follows.

- $\mathcal{U}$  derives  $t^*$  secrets from available PIFs  $\subseteq \overrightarrow{PIF}$ . For the sake of brevity, we denote the PIFs by  $\{PIF_1, \dots, PIF_{t^*}\}$ .
- $\mathcal{U}$  computes  $seed_i = F(PIF_1)$  and  $s_i^* = H(seed_i)$  for  $i = 1, 2, \dots, t^*$ .
- $\mathcal{U}$  generates a new polynomial over  $Z_p$ .

$$p_1(x) = x^{n^*} + \sum_{i=1}^{n^*-t^*} aux_i \cdot x^{n^*-i},$$

where  $aux_i$  denotes the  $i$ -th element in  $aux$ .

- $\mathcal{U}$  solves Equation (1) over  $Z_p$  to obtain  $b_{t^*-1}, \dots, b_0$ .

$$\begin{cases} p_1(s_1^*) = b_0 + b_1 s_1^* + \dots + b_{t^*-1} s_1^{*t^*-1} \\ \dots \\ p_1(s_{t^*}^*) = b_0 + b_1 s_{t^*}^* + \dots + b_{t^*-1} s_{t^*}^{*t^*-1} \end{cases} \quad (1)$$

Solving Equation (1) amounts to finding a polynomial  $p_2(x) = b_0 + b_1 x + \dots + b_{t^*-1} x^{t^*-1}$  such that  $p_2(s_i^*) = p_1(s_i^*)$  for  $j = 1, 2, \dots, t^*$ .

- $\mathcal{U}$  can utilize Lagrange interpolation to obtain  $p_2(x)$ .  $\mathcal{U}$  computes

$$w_i(x) = \prod_{\substack{1 \leq j \leq t \\ j \neq i}} \frac{x - s_j^*}{s_i^* - s_j^*},$$

$$p_2(x) = \sum_{i=1}^t p_1(s_i^*) \cdot w_i(x).$$

- $\mathcal{U}$  obtains  $p(x) = p_1(x) - p_2(x)$ .  $\mathcal{U}$  already has  $t^*$  roots of  $p(x)$  and can find other  $n^* - t^*$  roots, i.e.,  $\{s_{t^*+1}^*, \dots, s_{n^*}^*\}$ , using Berlekamp's algorithm [67].
- $\mathcal{U}$  computes the master key  $S = H(s_1^* || s_2^* || \dots || s_{n^*}^*)$ .

### 3.5 Deploying DRKM

DRKM is compatible with existing cryptosystems. In reality,  $\mathcal{U}$  has two methods to deploy DRKM as follows.

**Integrating with encryption.** For registered cryptosystems,  $\mathcal{U}$  can utilize  $S$  to encrypt a set existing cryptographic keys  $\vec{K}$  and outsource the ciphertexts to  $\mathcal{CS}$ . Subsequently,  $\mathcal{U}$  can access  $S$  with the aid of key servers or with  $t^*$  PIFs and decrypt the ciphertexts to obtain  $\vec{K}$ .  $\mathcal{U}$  can utilize private-key encryption or public-key encryption, and we discuss their pros and cons.

*Private-key encryption.* The computation costs of private-key encryption are much less than those of public-key one. In addition,  $\mathcal{U}$  can decrypt  $\vec{K}$  on demand when utilizing the CTR mode. However, when a user generates a new key,  $\mathcal{U}$  has to recover the master key  $S$  and then encrypt the new key.

*Public-key encryption.* Some public-key encryption algorithms, e.g., ElGamal encryption, support updating the ciphertexts without decryption when the public/private keys are updated (i.e., support for proxy re-encryption). Specifically, when  $\mathcal{U}$  updates  $S$ ,  $\mathcal{U}$  does not need to first decrypt  $\vec{C}$  with the previous private key and then encrypt  $\vec{K}$  with the newly-generated public key.  $\mathcal{U}$  can directly utilize the proxy re-encryption to update the ciphertexts of  $\vec{K}$ . What is more, when  $\mathcal{U}$  has a newly-generated key,  $\mathcal{U}$  can directly utilize the public key to encrypt the key without recovering  $S$ .

**Integrating with KDFs.** For a new cryptosystem, a more efficient method is to directly derive various new keys from  $S$  with mature KDFs [26]. In this case, the user does not need to perform encryption or decryption operations.

*Key rotation.* It is feasible for a sophisticated adversary to corrupt more than  $t^*$  PIFs given enough time [79]. Therefore, the master key should be updated after some PIFs are leaked.  $\mathcal{U}$  can re-execute the **Managing** algorithm in DRKM to rotate the master key using undisclosed PIFs. In this way, an adversary who compromises some previously used PIFs cannot get any information about the updated master key.

## 4 Security analysis

An adversary who compromises  $t - 1$  shares and  $t^* - 1$  PIFs cannot get any information about the master key. The security of DRKM is captured by Theorem 1, and we formally prove its security as follows.

**Theorem 1.** *Assuming  $H$  is a random oracle, a probabilistic polynomial-time (PPT) adversary who compromises any  $t - 1$  of  $n$  shares and any  $t^* - 1$  of  $n^*$  PIFs can distinguish the master key from a uniformly-chosen key from  $Z_p$  with probability no more than  $\frac{1}{2} + \text{negl}(\ell)$ , where  $\ell$  is the security parameter, and  $\text{negl}$  is a negligible function.*

*Proof.* We prove Theorem 1 with a series of games as follows.

**Game 0.** Assuming  $H$  is a random oracle, the **Setup** and **Managing** algorithms in DRKM is executed, where a master key  $S$  is derived from  $n^*$  PIFs and then

split into  $n$  shares. A key  $S^*$  is uniformly chosen from  $Z_p$ . To ensure consistency of  $H$ , the tuples  $(seed_1, s_1^*), (seed_2, s_2^*), \dots, (seed_{n^*}, s_{n^*}^*)$ , and  $(s_1^* || \dots || s_{n^*}^*, S)$  are recorded. A PPT adversary  $\mathcal{A}$  is given any  $t - 1$  of  $n$  shares,  $t^* - 1$  of  $n^*$  PIFs, and the auxiliary information  $aux$ . In other words,  $\mathcal{A}$  knows  $t^* - 1$  tuples of  $H$ , assuming the tuples  $(seed_1, s_1^*), (seed_2, s_2^*), \dots, (seed_{t^*-1}, s_{t^*-1}^*)$  for brevity.  $\mathcal{A}$  can query the random oracle at most  $q(\ell)$  times. Given  $S$  and  $S^*$ , if  $\mathcal{A}$  can distinguish them, then this game outputs 1, else outputs 0. The success probability of  $\mathcal{A}$  is  $\Pr[\text{Game } 0_{\mathcal{A}}(\ell) = 1] = \frac{1}{2} + \text{Adv}_0$ , where  $\text{Adv}_0$  denotes the advantage of  $\mathcal{A}$  in this game. We will prove that  $\text{Adv}_0$  is negligible.

**Game 1.** This game is the same with **Game 0** with the exception of one difference. The random oracle only records the tuples  $(seed_1, s_1^*), (seed_2, s_2^*), \dots, (seed_{t^*-1}, s_{t^*-1}^*)$  which are given to  $\mathcal{A}$  and  $(s_1^* || \dots || s_{n^*}^*, S)$ . For  $\mathcal{A}$ 's queries that are not recorded, the random oracle uniformly chooses  $s^* \leftarrow Z_p$ , and if  $s^* \in \{s_{t^*}^*, \dots, s_{n^*}^*\}$ , then re-chooses a random number until  $s^* \notin \{s_{t^*}^*, \dots, s_{n^*}^*\}$ . It returns  $s^*$  and records the corresponding tuple.  $\text{Adv}_1$  denotes the advantage of  $\mathcal{A}$  in this game.

For **Game 0** and **Game 1**,  $\mathcal{A}$  can win if and only if one of its queries has been recorded by the random oracle or it guesses correctly. Except for the tuples that  $\mathcal{A}$  knows, the oracle in **Game 0** records  $(n^* - t^* + 1)$  tuples, and the oracle in **Game 1** records one tuple. Therefore, we have

$$\text{Adv}_0 \leq (n^* - t^* + 1)\text{Adv}_1.$$

**Game 2.** This game is the same with **Game 1** with one difference. The master key  $S$  and each element of  $aux$  are uniformly chosen from  $Z_p$ . Thus, the random oracle only records  $t^* - 1$  tuples that are given to  $\mathcal{A}$ . At this point, the advantage that  $\mathcal{A}$  can win is almost zero, as Shamir's threshold secret sharing scheme is perfectly secure. Thus, we have  $\text{Adv}_1 - \text{Adv}_2 \leq \frac{q(\ell)}{2^\ell}$ .

To sum up, we have

$$\text{Adv}_0 \leq \frac{(n^* - t^* + 1)q(\ell)}{2^\ell},$$

i.e., an adversary cannot get any extra information about the master key even though it has compromised any  $t - 1$  of  $n$  shares and any  $t^* - 1$  of  $n^*$  PIFs. This concludes the proof.  $\square$

## 5 Implementation and evaluation

We implement a DRKM prototype and conduct a comprehensive performance evaluation. All experiments are conducted on a laptop with an Intel Core i5 processor running at 2 GHz using four cores and 16 GB DDR3 of RAM. Our source code is available on

<https://github.com/DRKM-code/DRKM.git>.

We evaluate the performance from the following aspects.



| PIF                | Computation delay (ms) |
|--------------------|------------------------|
| Fingerprint [25]   | $\approx 1.60$         |
| Face [5]           | $\approx 1.50$         |
| Iris [30]          | $\approx 3.00$         |
| Hand geometry [14] | $\approx 7.6$          |
| Palmprint [12]     | $\approx 6$            |
| ECG [47]           | $\approx 0.32$         |
| Hand gesture [35]  | $\approx 90 - 110$     |
| SIM card           | $\approx 0.003$        |
| Hardware token     | $\approx 1.36$         |
| Intel SGX          | $\approx 0.56$         |

Table 1: Computation delay of deriving secrets from PIFs.

### 5.1 Deriving secrets from PIFs

We present the computation delay to derive reconstructable secrets from different PIFs, as shown in Table 1.

**Someone the user is.** We investigate existing biometric feature recognition and extraction methods. A minutiae-based fingerprint individuality model [25] is utilized to extract features from users’ fingerprints. We leverage the scheme [5] for face recognition to extract the face features. We utilize the algorithm in [30] for iris recognition to extract the features of irises. The extracted iris features can be formatted with the biometric standards. We utilize the hand geometry recognition scheme [14], the palmprint recognition scheme [12], and the hand gesture recognition scheme [35] to extract the hand characteristics, respectively. We utilize Deep-ECG [47], a CNN-based biometric approach for ECG signals, to extract the features of ECG.

**Something the user has.** We simulate the authentication algorithms of a SIM card, hardware token, and Intel SGX on a laptop, and evaluate the performance. The hardware PIFs are dependent on stateful devices. Recalling Section 2, we can derive a reconstructable secret from a device-dependent PIF with the aid of another reconstructable secret.

### 5.2 Key derivation and management

We investigate existing authentication schemes and choose the applicable methods to derive secrets from PIFs. The computation delay for deriving reconstructable secrets from different PIFs is shown in Section 5.1. If  $n$  is larger than 10,  $\mathcal{U}$  can derive multiple instances from a type of PIF. For example, if  $\mathcal{U}$  needs to derive 20 reconstructable secrets, she/he can choose 2 irises, 2 hand geometries, 2 Palmprints, 1 ECG, 2 SIM cards, 1 hardware token, and 10 fingerprints. In the following, we assume that  $\mathcal{U}$  has obtained  $n^*$  secrets.

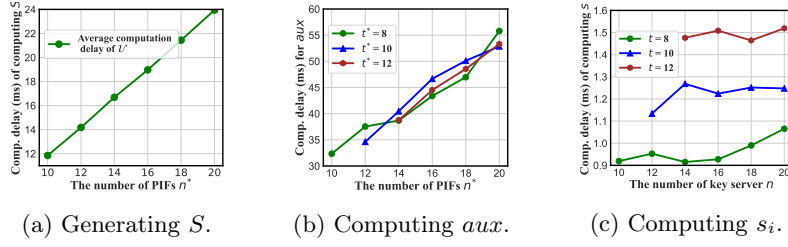
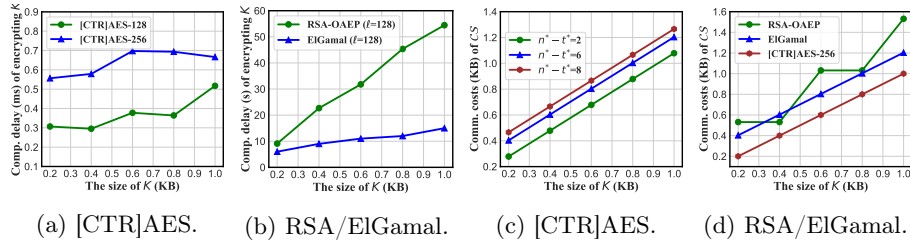
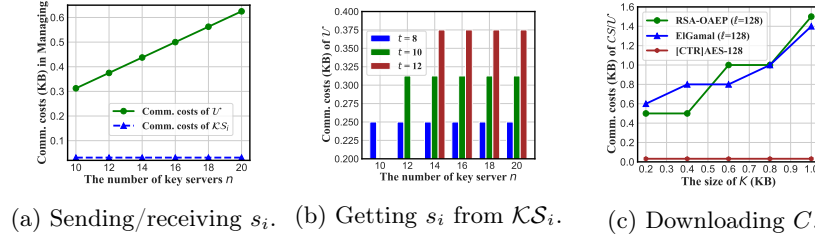
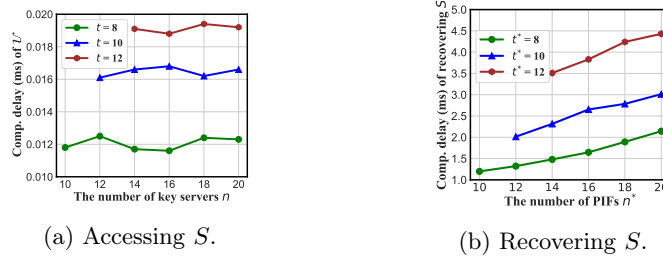
Fig. 4: The computation delay of  $\mathcal{U}$  in the **Managing** phase.Fig. 5: The costs of  $\mathcal{U}$  using different encryption schemes.

Figure 4(a) shows the computation delay on  $\mathcal{U}$  of deriving  $S$  from  $n^*$  secrets, where we set different  $n^*$ . The computation delay is an average of deriving 10 master keys.

$\mathcal{U}$  computes the auxiliary information  $aux$  which assists  $\mathcal{U}$  in recovering the master key from the PIFs. Figure 4(b) shows the computation delay on  $\mathcal{U}$  of  $aux$  with different  $n^*$  and  $t^*$ . The delay of computing  $aux$  decreases with  $t^*$ .  $\mathcal{U}$  shares  $S$  among the key servers in a threshold way.  $\mathcal{U}$  first computes  $n$  secret shares  $\{s_1, \dots, s_n\}$ . Figure 4(c) shows the delay in computing the secret shares.

$\mathcal{U}$  can use private-key encryption or public-key encryption to encrypt  $\vec{K}$ . Figure 5(a) and 5(b) show the computational delay for encrypting different sizes  $K$  with different encryption algorithms, including RSA-OAEP [18], ElGamal [33], and [CTR]AES. The computation delay for private-key encryption is much less than that for public-key encryption.

In DRKM,  $\mathcal{U}$  needs to send the secret shares to the key servers and outsource the auxiliary information and the ciphertexts to  $\mathcal{CS}$ . In Figure 6(a), we show the communication costs on  $\mathcal{U}$  and  $\mathcal{KS}_i$ . As shown in Figure 5(c), the communication costs on  $\mathcal{CS}$  are approximate to the size of  $K$  when using private-key encryption. For the same size  $K$ , the communication costs on  $\mathcal{CS}$  increase with  $n^* - t^*$ , because the larger  $n^* - t^*$  is, the larger size of  $aux$  is when recovering  $S$ . Figure 5(d) shows the communication costs of  $\mathcal{CS}$ . The communication costs when using public-key encryption are significantly larger than those when using [CTR]AES.

Fig. 6: The communication costs of  $\mathcal{U}$ ,  $\mathcal{KS}_i$ , and  $\mathcal{CS}$ .Fig. 7: The computation delay of  $\mathcal{U}$ .

### 5.3 Key access and recovery

When  $\mathcal{U}$  needs to access the master key,  $\mathcal{U}$  interacts with the key server via the secure channels to get  $t$  secret shares. In Figure 6(b), we show the communication costs on  $\mathcal{U}$  and each key server with different  $n$  and  $t$ .  $\mathcal{U}$  also needs to download the ciphertexts from  $\mathcal{CS}$ .  $\mathcal{U}$  can download the ciphertexts on demand when using private-key encryption. Figure 6(c) shows the communication costs of  $\mathcal{CS}$  and  $\mathcal{U}$ , where  $\mathcal{U}$  only accesses one key. Figure 7(a) shows the delay on  $\mathcal{U}$  to compute  $S$  from  $t$  secret shares.

If the number of available key servers is less than  $t$ ,  $\mathcal{U}$  has to recover the master by utilizing available PIFs.  $\mathcal{U}$  needs to get at least  $t^*$  secrets from available PIFs. We assume that  $\mathcal{U}$  has derived  $t^*$  secrets. Figure 7(b) shows the delay on  $\mathcal{U}$  to recover  $S$  from  $t^*$  secrets and  $aux$ . The results show that the delay of recovering  $S$  from secrets is less than 5 ms.

## 6 Comparisons and applications

### 6.1 Comparison with KDF

The core of DRKM is to convert a user's PIFs to a reconstructable master key which is utilized to encrypt the user's keys used in various cryptosystems. A natural method to transform a PIF into a key is key derivation functions (KDF). In this subsection, we compare DRKM with different KDFs and further elaborate on the desirable advantages of DRKM.

| Method                    | Computation delay (ms) |
|---------------------------|------------------------|
| DRKM (Before destruction) | 0.012                  |
| DRKM (After destruction)  | 90.72                  |
| Threshold MFKDF           | 89.82                  |

Table 2: Comparison with MFKDF. Set  $t = t^* = 8$  and  $n = n^* = 10$ .

We start with password-based key derivation functions (PBKDFs) [19,43] that derive keys from users’ passwords. The proliferation of PBKDFs provides protection for users without introducing key management problems. Typically, PBKDF1 and PBKDF2 [43] are widely used in network applications, e.g., WPA2 [38] protocols in wireless communication systems. However, the schemes based on PBKDFs are generally confronted with dictionary guessing attacks, since passwords are inherently low-entropy. Many recent security incidents have shown that utilizing passwords as the sole defense line is indeed insufficient.<sup>14</sup>

Nair et al. [55] proposed a threshold multi-factor key derivation function (TMFKDF), which allows a user to derive a key from  $n^*$  PIFs, and recover the key from any  $t^* - 1$  of them. We compare DRKM with TMFKDF as follows.

Regarding costs, Table 2 shows the comparison between DRKM and TMFKDF in terms of computation delay of accessing the master key with  $t = t^* = 8$  and  $n = n^* = 10$ . The results show that in normal times, the delay using DRKM is 5~6 orders of magnitude less than that of using TMFKDF.

Regarding functionality, the key recovery provided by TMFKDF depends on the availability of metadata, i.e., encrypted shares. Users cannot recover their keys without metadata even if they utilize  $n^*$  storage-independent PIFs. In DRKM, users can directly retrieve their keys from  $n^*$  PIFs without any storage, if all  $n^*$  PIFs are independent of storage.

Regarding security and practicality, the shares in DRKM are independent of PIFs used for key derivation, while in [55], a share is protected by a PIF. In DRKM, an adversary can compromise any  $t - 1$  shares and any  $t^* - 1$  PIFs, but in [55], it is only allowed to obtain any  $t^* - 1$  PIFs and the *corresponding* shares. DRKM enables users to balance security and efficiency but the TMFKDF fails to achieve it. Specifically, there are two independent thresholds in DRKM. The threshold in TMFKDF corresponds to  $(t^*, n^*)$  in DRKM. Once an adversary compromises  $t^*$  shares (i.e., PIFs) of TMFKDF, it can recover the key. In DRKM, such an adversary cannot get any information about the master key, since the user can set  $t \geq t^*$ . The user can achieve a trade-off between security and efficiency by adjusting  $(t, n)$  while remaining  $(t^*, n^*)$ .

## 6.2 Comparison with alternative schemes

Currently, some products and solutions for secure and reliable key management have been proposed, and they might be trivially extended to achieve destruction

<sup>14</sup> <https://www.cybersecuritydive.com/news/paypal-credential-stuffing-attack/640804>

resistance. We will discuss them in detail, analyze their inherent problems, and compare them with DRKM as follows. Our aim in presenting this section is twofold. The first one is to show the advantages of DRKM in terms of security, functionality, and efficiency. The second one is to demonstrate that designing usable and destruction-resistant key management schemes is very challenging. Generally, existing key management schemes can be categorized into two types: fully local management ones and fully outsourcing management ones.

**Fully local management schemes.** A user keeps her/his keys in well-guarded devices and keeps them in safe places. Notable examples include YubiKey [77] and Ledger [49]. To achieve destruction resistance, the user can simultaneously utilize YubiKey and Ledger to store the same key (one for general use and the other for backup) and keep them in different secure locations.

However, the disadvantage of the above scheme is obvious. The user needs to ensure the security and reliability of the backup key, which requires the user to continuously monitor its condition to minimize the possibility of undetected leakage. Actually, this would cause prohibitive costs for the user. As an empirical observation, individuals do not always have a secure secondary location to store the backup key.

**Fully outsourcing management scheme.** A user requests key management services from service providers. Typical examples include Keywhiz [45], where the user’s key is split into  $n$  shares in a threshold way and let each key server store one of them. The user can recover her/his key by interacting with any  $t$  of  $n$  key servers. Compared with a fully local management scheme, this scheme achieves portability: it enables the user to recover the key as needed without maintaining any secret on local devices. It achieves destruction resistance to some extent: as long as  $t$  key servers are available, the key can be recovered.

Obviously, the destruction resistance provided by this scheme relies on a strong assumption that at least  $t$  key servers would not be destroyed. This problem cannot be trivially addressed by requiring the key server to back up the share, as it increases the danger of security breaches significantly [65].

**DRKM: a hybrid “local + outsourcing” key management scheme.** In DRKM, a user derives a master key from her/his PIFs and shares it among the key servers in a threshold way. DRKM inherits the advantages of the above schemes: in normal times, the user can access the master key by interacting with the key servers; and once all key servers and the user’s devices are destroyed, the master key can be recovered from a part of PIFs used for key derivation. DRKM also overcomes their drawbacks: compared with the fully local management scheme, DRKM only requires a *stateless* device and a small number of PIFs to recover the master key after the destruction occurs; compared with the fully outsourcing management one, users in DRKM can retrieve their keys *per se*, even if all key servers are unavailable.

### 6.3 Applications and compatibility

With storage/device-independent PIFs, we can easily construct a destruction-resistant threshold multiple-factor authentication scheme, where a user only

needs to register with a service provider by more than a threshold number of storage/device-independent PIFs. After the destruction occurs, the user can still log in to the service provider via available PIFs. We further emphasize the practical nature of DRKM as well as DRKM-based DRC (i.e., DRKM + commercial backup system) in the following.

One potential application is to manage a secret—say, a key to a bank’s vault—that is shared among a board of directors, where the vault is protected by cryptosystems. Anytime when the vault needs to be opened, it should be confirmed and agreed by a majority of members. With DRKM, each member can contribute the secret using her/his PIFs with the  $(t^*, n^*)$ -secret aggregation with threshold retrieval mechanism, and then the secret is shared among multiple key servers with the  $(t^*, n^*)$ -secret sharing scheme. Each key server is available for a specific member. Before device destruction occurs, the members can recover the secret by interacting with their key servers in a portable way. Once the number of the available key servers is less than  $t^*$ , they can cooperatively recover the secret using their PIFs. We stress that regarding device destruction and security, neither Shamir’s secret sharing scheme nor the threshold password-hardening protocols [65,17,79,39,10,78,48,41,27] can achieve it.

Another promising application of DRKM and DRKM-based DRC is to resist ransomware attacks. We notice that the success of ransomware attacks is to make devices unavailable to their users. As such, if the maintenance of the most sensitive information (e.g., the master key) does not rely on any device, a ransomware attack will be doomed to failure (we implicitly require a backup system to store the non-sensitive information, such as ciphertexts, signatures). With DRKM, a user can derive a master key from her/his PIFs and maintain the master key locally or using a set of key servers. Once the master key is hacked by ransomware attacks, the user can recover the master key from the PIFs that are used to derive it. Therefore, DRKM-based DRC can be directly deployed to thwart ransomware attacks.

Recently, many novel PIFs have been proposed, e.g., PCR-Auth [37] and Capacitive Plethysmogram [76]. We stress that DRKM has forward-compatibility with future PIFs. As long as a PIF satisfies Definition ?? and can be represented by a unique binary string, it can then be utilized in DRKM to derive a master key, which potentially applies to newly-discovered PIFs in the future.

## 7 Related work

### 7.1 Introduction to popular PIFs

Generally, users’ personal identification factors (PIFs) can be divided into three categories: something the user knows, someone the user is, and something the user has [22].

**Something the user knows** This type of PIFs binds a user’s identity with a secret only known to the user. As long as the user outputs the correct secrets, she/he can identify herself/himself.

- *Password*. A password is a character string chosen by a user and can be utilized to construct a portable authentication scheme, where the user takes her/his password as the sole input for authentication [10]. Password is memorable and thereby is destruction-resistant. Generally, a cryptographic hash function is used to “obfuscate” the password, and the hashed password serves as the authentication credential. However, the password is inherently low-entropy and is vulnerable to dictionary guessing attacks (DGA) [72].

- *PIN code*. A personal identification number (PIN) code is a set of numbers (which generally consists of four or six digits) generated by the user. PIN codes’ entropy is much lower than passwords. Therefore, PIN codes should not be utilized to serve as the sole PIF in the system [71].

- *Private key*. In public-key cryptosystems, a private key is a string uniformly chosen from some set (where the number of elements in the set should be large enough) and is used to generate a corresponding public key. With the pair of keys, an identification scheme can be constructed (e.g., the Schnorr identification scheme [64]). In reality, a private key is always stored in a stateful device or hardware security module (as it is high-entropy), and thereby cannot be recovered if the device (or hardware module) is destroyed.

**Someone the user is** This type of PIFs is essentially the characteristics that can uniquely identify the user. Such a PIF is not necessarily maintained in some device. As a consequence, most of these PIFs (discussed in this paper) are destruction-resistant.

A general procedure of biometric characteristic recognition is described as follows. A user collects biometric characteristics with the aid of stateful devices (e.g., obtaining a face image by using a camera), and then some features can be extracted from the characteristics. With the features, some templates can be derived to serve as authentication credentials. We investigate popular biometric PIFs and feature extraction methods. Subsequently, a “fresh” template can be extracted from the user’s biometric characteristics to be compared with the pre-generated templates for user authentication. The widely-used biometric characteristics, e.g., fingerprints and faces, can be easily collected using smartphones. With the proliferation of wearable devices, it is also convenient to collect other biometric characteristics, e.g., ECGs [8]. Typical biometric characteristics are introduced in the following.

- *Fingerprint*. Fingerprints are the most widely utilized biometric characteristic for user authentication. A fingerprint can be transformed into a unique image with ridges and valleys, where a ridge is a single curved segment, and a valley is a region between two adjacent ridges, and minutiae-based feature extraction is the most commonly-used method in fingerprint recognition [70,24]. For users, it is convenient to utilize fingerprints to authenticate herself/himself, since most devices have fingerprint recognition functions. Fingerprints serving as PIF achieve compromise resilience, since the user can use different fingerprints in different systems. However, it is not so hard to collect targeted users’ fingerprints in the physical world [75].

- *Face*. Facial recognition is also widely used for user authentication. As facial recognition only requires a camera to collect the face image, most smartphones utilize the face as the PIF. In practice, extraction methods based on either geometrical features or statistical features [74] can be utilized to extract features from users' faces. Compared with fingerprints, users' faces fail to achieve compromise resilience.

- *Iris*. An iris camera captures a user's pupil and extracts the iris image from the pupil by using Gabor filters. Compared with fingerprint-based and face-based authentication, iris-based authentication is more accurate for identifying users [61]. Although irises also fail to achieve compromise resilience, collecting users' irises is much harder than collecting fingerprints and faces for adversaries. However, iris recognition requires a specific-purpose device to scan the iris, which causes high costs to deploy the iris-based authentication.

- *Hand*. In reality, the physical characteristics of a hand (e.g., hand geometry [21], palmprints [36], and hand gestures [68]) can also serve as PIFs for user authentication, where hand geometry and hand gestures can be utilized to achieve non-contact recognition. The accuracy rate is significantly affected by the environment, e.g., lighting, and the devices collecting palmprints are generally larger, compared with those collecting fingerprints and faces.

- *Electrocardiographic (ECG)*. An ECG records the electrical activity of the heart and mainly consists of P wave, QRS complex, and T wave. ECG is highly personalized and can serve as PIF [56]. ECG-based authentication requires specific types of equipment, e.g., ECG monitors.

**Something the user has** This type of PIFs binds a user's identity with a stateful device or hardware unit. Anyone who possesses the device (or the unit) can pass authentication.

- *Subscriber identity module (SIM) card*. A SIM card is an integrated circuit card, that has been utilized in cellular networks, e.g., 3G and 4G [50]. The authentication protocol is based on message authentication code (MAC) [15]. Specifically, a SIM card stores a universal international mobile subscriber identity (IMSI) number and a 128-bit key that is shared with a server (deployed by the communication service provider). The user and the server initialize the same sequence number  $SQN$ . A typical SIM-based authentication procedure is shown in Figure 1, which follows a challenge-response paradigm. When subscribing to cellular networks, the user sends her/his IMSI number as  $ID$  to the server. The server first generates a challenge message, including a random number  $RAND$  and a MAC for  $RAND$  and  $SQN$ , and sends it to the user. The user then verifies the received MAC and generates a response message by computing a MAC for  $RAND$ . The server finally verifies the MAC and allows the user to subscribe if it is valid. In Figure 1, the MAC is based on pseudorandom functions  $f1(\cdot)$ ,  $f2(\cdot)$ , and  $f5(\cdot)$ , and are instantiated by AES in practice [34].

With the proliferation of mobile devices, e.g., smartphones, SIM cards are widely used for authentication in daily life, which is convenient for users. This



also implies that the utilization of SIM cards has to rely on devices with a cellular communication module.

- *Hardware token.* Hardware tokens can be divided into two categories from the point of the underlying cryptosystem: symmetric-key-based ones and public-key-based ones.

For symmetric-key-based hardware tokens, typical examples include RSA SecurID [63], which is constructed on a hash-based MAC (HMAC). A simplified authentication procedure is shown in Figure 2. The hardware token stores a seed that is shared with a server. During the sign-on phase, the hardware token and the server invoke a one-time password algorithm (which is based on an HMAC) using the seed and current time as input [54]. The user sends the password generated by the hardware token to the server. The server verifies whether the received password is the same as the locally generated one. If so, the user passes the authentication.

For public-key-based hardware tokens, typical examples include U2F token [31], which is based on ECDSA [42]. Figure 3 shows the authentication procedure. The hardware token stores a private key, and the corresponding public key is stored on the server. In the sign-on phase, the server sends a nonce *Nonce* as a challenge to the hardware token, and the hardware token signs *Nonce* and returns the signature to the server. The server verifies the signature with the corresponding public key. If it is valid, the user passes the authentication.

It is difficult to forge a valid hardware token. However, once adversaries physically access the hardware token, they can easily impersonate the user to pass authentication.

- *Trusted execution environment (TEE).* TEEs are isolated private enclaves inside CPU, which are used to protect data. Intel Software Guard Extensions (Intel SGX) is the most widely-used TEE architecture [29]. It can also be utilized to authenticate users by using its two root keys: the root provisioning key (RPK) and the root seal key (RSK), where RPK and RSK are fused in CPU by the manufacturer. Due to the space limitation, please refer to Ref. [29,13] for more details. In addition, except Intel SGX, other TEEs, e.g., ARM TrustZone [58], are also widely used for authentication.

Such hardware units rely on specific manufacturers and are not easily forged, but they may suffer from side-channel attacks [23]. Users and servers have to equip the same TEEs.

## 7.2 Key management schemes

Many key management schemes have been proposed in the past few years. We discuss them in the following.

**Hardware-based key management.** Users can locally manage their keys by utilizing secure hardware devices. For example, Intel Software Guard Extensions (SGX), aiming to protect the confidentiality and integrity of computations on sensitive data performed on a computer, can also be utilized for key management. Priebe et al. proposed a secure database using SGX [60], which can be

utilized for key management. However, such a scheme requires the user to possess a device equipped with Intel central processing units (CPUs) that support Intel SGX. In addition, Intel SGX is vulnerable to side-channel attacks [23].

Another approach to key management is to generate keys based on the physical characteristics of a hardware device rather than storing them within it. Specifically, although the manufacturing process is the same among different ICs, each IC is actually different from the others, which is called manufacturing variability and can be utilized for key generation. For example, Maes et al. proposed physical unclonable functions (PUFs) to derive keys by leveraging the variability [52].

Hardware-based key management schemes essentially shift the problem of managing multiple keys from protecting each individual key to protecting another key that can unlock them. Once an adversary physically accesses the hardware devices, he may recover the user's keys. To resist such an adversary, a widely-used remedy is to introduce an additional authentication mechanism: only the user who passes the authentication can utilize the hardware devices. For example, before using a SIM card, the user needs to input a PIN code to unlock it. The user needs to input the correct password before utilizing the hardware token for authentication [69]. MacOS provides a secure container, called Keychain, which assists users in managing their keys and passwords [4]. MacOS authenticates users by using passwords, fingerprints, or faces, and only authenticated users can access the Keychain.

Whereas, the functionality and security of hardware-based key management schemes depend on the reliability of the hardware devices. Once the hardware devices are destroyed (e.g., due to wrong formatting), the user would never recover her/his keys. Especially, the hardware devices are individually maintained by the users and are vulnerable to being lost or stolen. In addition, hardware-based key management schemes fail to achieve portable key access.

**Software (or extended services) based key management.** To free the costs and issues introduced by local key management, users prefer to employ a service provider to achieve key management. For instance, Amazon provides key management services for users [1], where a master key is generated by each user and utilized to encrypt other keys. The master key and the ciphertexts of other keys are well maintained by Amazon. Microsoft Azure [3] and Google Cloud Platform (GCP) [2] provide the same key management services for users. With the assistance of the service providers, the users are able to manage and access their keys on any device. Nevertheless, for these key management services, service providers rely on hardware security modules (HSMs) to manage users' master keys, e.g., Amazon utilizes a distributed fleet of FIPS 140-2 validated HSMs to securely manage the users' master keys [1]. Once the HSMs are destroyed for some reason (e.g., servers are destroyed due to geological disasters), the users' master keys would no longer be recovered.

To eliminate the reliance on security hardware devices, server-aided key management schemes have been proposed in the past few years. A user utilizes a master key to encrypt other keys, and the master key is split among key servers

in a threshold way [11,20]. As long as the number of available key servers is larger than the threshold, the user can recover the key. An adversary who compromises fewer than a threshold number of key servers cannot obtain anything about the master key. A notable example is Keywhiz, an open-source distributed key management software [45].

To achieve portability, password-hardening protocols [17,79,39,10,78,48,41,27] can also be utilized for key management, where a user hardens her/his password with the aid of the key servers in a threshold and oblivious way. After being hardened, the password is secure against dictionary guessing attacks (DGA), which enhances the security significantly. The hardened password is utilized to compute authentication credentials and generate the master key. Such a password-hardening-based key management scheme is destruction-resistant, since the password is memorable, and any destruction of the user's devices would not cause the unavailability of the password. Whereas, it is still unsatisfactory in real-world deployment, since the password serves as the sole secret. Once the password is compromised, no security is guaranteed. In reality, compromising users' passwords is not so hard for a sophisticated adversary, even if they are hardened by the key servers [72,73].

In addition to directly sharing the master key among multiple key servers, distributed encryption schemes have been proposed as a variant of distributed key management schemes. Agrawal et al. proposed the first formal threshold symmetric-key encryption based on distributed pseudorandom functions in DiSE [11], where the user employs a group of key servers to generate an encryption key for each message. Specifically, the user generates a commitment to a message and sends the commitment to each server, and each server computes a response utilizing its key share for the user. The user aggregates a threshold number of responses and gets the message-specific key to encrypt the message. However, DiSE may introduce heavy computation and communication costs to the user when encrypting a large set of messages. Christodorescu et al. proposed an amortized threshold symmetric-key encryption scheme [28], which enables the user to encrypt a large set of messages using a single interaction. In addition, Jarecki et al. constructed an oblivious key management system based on oblivious pseudorandom functions [40]. This scheme has an updatable encryption capability, and the update procedure of the ciphertexts does not require the user to interact with the key servers.

Whereas, in the above schemes, sophisticated adversaries may corrupt enough key servers given enough time and get the users' master key. To resist such adversaries, multiple proactivization mechanisms for secret sharing have been proposed [16,53]. In these schemes, time is divided into fixed intervals called epochs, and the secret shares are updated with the new ones without changing the master key in different epochs. To further enhance security, the key servers can be replaced by the newly employed ones in different epochs while maintaining the master key.

These server-aided key management schemes are free from reliance on secure hardware devices. However, these schemes cannot resist the destruction of key

servers. The functionality and reliability totally rely on whether the key servers can provide services for users. Once the key servers are destroyed, the user would never recover their keys.

### 7.3 Portable authentication

Biometric characteristics provide a convenient and portable way for users to authenticate themselves with servers. Despite the benefits of using biometric characteristics, there exist security issues. Specifically, storing a user's biometric characteristic template on the server side as an authentication credential makes the user's biometric characteristic vulnerable if the credential database is leaked. Furthermore, an attacker may perform trawling attacks to obtain authentication credentials based on different biometric characteristics from different authentication systems. This can enable the attacker to retrieve the user's master key if the user utilizes the same characteristics when deploying DRKM.

A method to mitigate the above attacks is to require the server to store the encrypted templates. The adversary cannot get anything about users' biometric characteristics from the compromised credential database. However, this method cannot resist the internal adversary, e.g., the malicious insiders working at the server, who can still get the users' templates.

Recently, the authentication protocol, FIDO U2F [69], has been proposed, which enables users' biometric templates to be stored on the users' devices instead of the server side. Specifically, the user installs an FIDO authenticator on her/his device. During the registration phase, the user unlocks the FIDO authenticator using fingerprint or other biometric characteristics and generates a new public/private key pair. The public key is sent to the server, and the private key and the biometric characteristic templates are stored on the device. During the sign-on phase, the server sends a challenge message, e.g., a nonce, to the user. The user unlocks the FIDO authenticator using the same biometric characteristics as those utilized in the registration phase and signs the challenge message using the private key. The user sends the signature back to the server, and the server verifies it with the stored public key. If it is valid, the user passes the authentication. As such, deploying FIDO authenticators can avoid the leakage of biometric information.

## 8 Conclusion

In this paper, we have investigated popular personal identification factors (PIFs) and proposed three concepts (i.e., storage-independent PIFs, device-independent PIFs, and device-dependent PIFs), and given the categorization criteria. We have proposed a series of methods to derive reconstructable secrets from a special class of device-dependent PIFs in tandem with storage/device-independent ones. We have constructed DRKM, a destruction-resistant key management scheme with portability. We have formally proven the security of DRKM. We have implemented a DRKM prototype and conducted a comprehensive performance evaluation to demonstrate its high efficiency.

For the future work, we will explore new storage-independent PIFs with high min-entropy. Furthermore, with such PIFs, it may be possible to construct a key management scheme that simultaneously achieves destruction resistance and portability after the destruction occurs.

## References

1. AWS Key Management Service: Developer Guide. <https://docs.aws.amazon.com/pdfs/kms/latest/developerguide/kms-dg.pdf>
2. Cloud Key Management. <https://cloud.google.com/security-key-management/#section-3>
3. Key Management in Azure. <https://learn.microsoft.com/en-us/azure/security/fundamentals/key-management>
4. Keychain Services. [https://developer.apple.com/documentation/security/keychain\\_services](https://developer.apple.com/documentation/security/keychain_services)
5. FingerTec Face Recognition Technology White Paper. <https://www.fingertec.com/download/tips/whitepaper-02.pdf> (2009)
6. Edward Snowden: Timeline. <https://www.bbc.com/news/world-us-canada-23768248> (2013)
7. Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region. <https://aws.amazon.com/cn/message/41926/> (2017)
8. <https://www.cbsnews.com/essentials/the-best-smart-watches-for-heart-health-monitoring-2023-08-23> (2023)
9. Agrawal, N.: Amazon Cloud Service Outage Breaks Parts of the Internet. <https://www.latimes.com/business/technology/la-fi-tn-amazon-service-outage-20170228-story.html> (2017)
10. Agrawal, S., Miao, P., Mohassel, P., Mukherjee, P.: PASTA: Password-Based Threshold Authentication. In: Proc. ACM CCS. pp. 2042–2059 (2018)
11. Agrawal, S., Mohassel, P., Mukherjee, P., Rindal, P.: DiSE: Distributed Symmetric-Key Encryption. In: Proc. ACM CCS. pp. 1993–2010 (2018)
12. Ahmadi, M., Soleimani, H.: Palmprint Image Registration Using Convolutional Neural Networks and Hough Transform. arXiv preprint arXiv:1904.00579 (2019)
13. Anati, I., Gueron, S., Johnson, S., Scarlata, V.: Innovative Technology for CPU Based Attestation and Sealing. In: Proc. HASP. vol. 13 (2013)
14. Angadi, S., Hatture, S.: Hand Geometry Based User Identification Using Minimal Edge Connected Hand Image Graph. IET Comput. Vision **12**(5), 744–752 (2018)
15. Arkko, J., Lehtovirta, V., Eronen, P.: Improved Extensible Authentication Protocol Method for 3rd Generation Authentication and Key Agreement (EAP-AKA'). Tech. rep. (2009)
16. Basu, S., Tomescu, A., Abraham, I., Malkhi, D., Reiter, M.K., Sirer, E.G.: Efficient Verifiable Secret Sharing with Share Recovery in BFT Protocols. In: Proc. ACM CCS. pp. 2387–2402 (2019)
17. Baum, C., Frederiksen, T., Hesse, J., Lehmann, A., Yanai, A.: PESTO: Proactively Secure Distributed Single Sign-on, or How to Trust a Hacked Server. In: Proc. EuroS&P. pp. 587–606 (2020)
18. Bellare, M., Rogaway, P.: Optimal Asymmetric Encryption. In: Proc. EURO-CRYPT. pp. 92–111 (1994)
19. Biryukov, A., Dinu, D., Khovratovich, D.: Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. In: Proc. IEEE EuroS&P. pp. 292–302 (2016)

20. Blakley, G.R.: Safeguarding Cryptographic Keys. In: Managing Requirements Knowledge, International Workshop on. IEEE Computer Society. pp. 313–313 (1979)
21. Boukhayma, A., Bem, R.d., Torr, P.H.: 3D Hand Shape and Pose from Images in the Wild. In: Proc. CVPR. pp. 10843–10852 (2019)
22. Brainard, J., Juels, A., Rivest, R.L., Szydlo, M., Yung, M.: Fourth-Factor Authentication: Somebody You Know. In: Proc. ACM CCS. pp. 168–178 (2006)
23. Chen, G., Chen, S., Xiao, Y., Zhang, Y., Lin, Z., Lai, T.H.: SGXPECTRE: Stealing Intel Secrets from SGX Enclaves via Speculative Execution. In: Proc. IEEE S&P. pp. 142–157 (2019)
24. Chen, H., Rouhani, B.D., Fu, C., Zhao, J., Koushanfar, F.: Deepmarks: A Secure Fingerprinting Framework for Digital Rights Management of Deep Learning Models. In: Proc. ACM ICMR. pp. 105–113 (2019)
25. Chen, J., Moon, Y.S.: A Minutiae-Based Fingerprint Individuality Model. In: Proc. IEEE CVPR. pp. 1–7 (2007)
26. Chen, L.: Recommendation for Key Derivation Using Pseudorandom Functions. NIST SP 800-108r1 (2022)
27. Chen, L., Li, Y.N., Tang, Q., Yung, M.: End-to-Same-End Encryption: Modularly Augmenting an App with an Efficient, Portable, and Blind Cloud Storage. In: Proc. USENIX Security. pp. 2353–2370 (2022)
28. Christodorescu, M., Gaddam, S., Mukherjee, P., Sinha, R.: Amortized Threshold Symmetric-Key Encryption. In: Proc. ACM CCS. pp. 2758–2779 (2021)
29. Costan, V., Devadas, S.: Intel SGX Explained. Cryptology ePrint Archive (2016)
30. Daugman, J.: How Iris Recognition Works. In: The Essential Guide to Image Processing, pp. 715–739 (2009)
31. Dauterman, E., Corrigan-Gibbs, H., Mazières, D., Boneh, D., Rizzo, D.: True2F: Backdoor-Resistant Authentication Tokens. In: Proc. IEEE S&P. pp. 398–416 (2019)
32. Dodis, Y., Reyzin, L., Smith, A.: Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data. In: Proc. EUROCRYPT. pp. 523–540 (2004)
33. ElGamal, T.: A Public Key Cryptosystem and A Signature Scheme based on Discrete Logarithms. IEEE Trans. Inf. Theory **31**(4), 469–472 (1985)
34. ETSI: Universal Mobile Telecommunications System (UMTS); LTE; 3G Security; Specification of the MILENAGE algorithm set: An Example Algorithm Set for the 3GPP Authentication and Key Generation Functions f1, f1\*, f2, f3, f4, f5 and f5\*, 3GPP TS 35.205 version 10.0.0 Release 10 (2011)
35. Fang, Y., Wang, K., Cheng, J., Lu, H.: A Real-Time Hand Gesture Recognition Method. In: Proc. ICME. pp. 995–998 (2007)
36. Han, Y., Tan, T., Sun, Z., Hao, Y.: Embedded Palmprint Recognition System on Mobile Devices. In: Proc. ICB. pp. 1184–1193 (2007)
37. Huang, L., Wang, C.: PCR-Auth: Solving Authentication Puzzle Challenge with Encoded Palm Contact Response. In: Proc. IEEE S&P. pp. 1034–1048 (2022)
38. IEEE: IEEE Standard for Information Technology: Telecommunications and Information Exchange between Systems, Local and Metropolitan Area Networks, Specific Requirements. Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 6: Medium Access Control (MAC) Security Enhancements. IEEE Std. 802.11i-2004 (2004)
39. Jarecki, S., Kiayias, A., Krawczyk, H., Xu, J.: TOPPSS: Cost-Minimal Password-Protected Secret Sharing Based on Threshold OPRF. In: Proc. ACNS. pp. 39–58 (2017)

40. Jarecki, S., Krawczyk, H., Resch, J.: Updatable Oblivious Key Management for Storage Systems. In: Proc. ACM CCS. pp. 379–393 (2019)
41. Jia, C., Wu, S., Wang, D.: Reliable Password Hardening Service with Opt-Out. In: Proc. SRDS. pp. 250–261 (2022)
42. Johnson, D., Menezes, A., Vanstone, S.: The Elliptic Curve Digital Signature Algorithm (ECDSA). *Int. J. Inf. Secur.* **1**(1), 36–63 (2001)
43. Kaliski, B.: PKCS# 5: Password-Based Cryptography Specification Version 2.0. Tech. rep. (2000)
44. Keelveedhi, S., Bellare, M., Ristenpart, T.: DupLESS: Server-Aided Encryption for Deduplicated Storage. In: Proc. USENIX Security. pp. 179–194 (2013)
45. Keywhiz: <https://square.github.io/keywhiz/> (2015)
46. Krawczyk, H.: Cryptographic Extraction and Key Derivation: The HKDF Scheme. In: Proc. CRYPTO. vol. 6223, pp. 631–648 (2010)
47. Labati Ruggero, D., Munoz, E., Piuri, V., Sassi, R., Scotti, F.: Deep-ECG Convolutional Neural Networks for ECG Biometric Recognition. *Pattern Recognit. Lett.* **126**, 78–85 (2019)
48. Lai, R.W., Egger, C., Reinert, M., Chow, S.S., Maffei, M., Schröder, D.: Simple Password-Hardened Encryption Services. In: Proc. USENIX Security. pp. 1405–1421 (2018)
49. Ledger: <https://www.ledger.com>
50. Lee, K., Kaiser, B., Mayer, J., Narayanan, A.: An Empirical Study of Wireless Carrier Authentication for SIM Swaps. In: Proc. SOUPS. pp. 61–79 (2020)
51. Legislature, C.: California Consumer Privacy Act of 2018 (as amended by the California Privacy Rights Act of 2020) (2020)
52. Maes, R., Leest, V.v.d., Sluis, E.v.d., Willems, F.: Secure Key Generation from Biased PUFs. In: Proc. CHES. pp. 517–534 (2015)
53. Maram, S.K.D., Zhang, F., Wang, L., Low, A., Zhang, Y., Juels, A., Song, D.: CHURP: Dynamic-Committee Proactive Secret Sharing. In: Proc. ACM CCS. pp. 2369–2386 (2019)
54. M’Raihi, D., Machani, S., Pei, M., Rydell, J.: Totp: Time-Based One-Time Password Algorithm. Tech. rep. (2011)
55. Nair, V., Song, D.: Multi-Factor Key Derivation Function (MFKDF) for Fast, Flexible, Secure, & Practical Key Management. In: Proc. USENIX Security (2023)
56. Odinaka, I., Lai, P.H., Kaplan, A.D., O’Sullivan, J.A., Sirevaag, E.J., Rohrbach, J.W.: ECG Biometric Recognition: A Comparative Analysis. *IEEE Trans. Inf. Forensics Secur.* **7**(6), 1812–1824 (2012)
57. Osborne, C.: Colonial Pipeline Ransomware Attack: Everything You Need to Know. <https://www.zdnet.com/article/colonial-pipeline-ransomware-attack-everything-you-need-to-know/>
58. Pinto, S., Santos, N.: Demystifying ARM TrustZone: A Comprehensive Survey. *ACM Comput. Surv.* **51**(6), 1–36 (2019)
59. Poonia, G.: The Reason not to Throw Away Old Hard Drives Might Be Surprising—There Could be Bitcoin on There. <https://www.deseret.com/2021/12/10/22827963/james-howells-threw-away-hard-drives-with-bitcoin-password> (2021)
60. Priebe, C., Vaswani, K., Costa, M.: EnclaveDB: A Secure Database Using SGX. In: Proc. IEEE S&P. pp. 264–278 (2018)
61. Proença, H., Neves, J.C.: IRINA: Iris Recognition (even) in Inaccurately Segmented Data. In: Proc. CVPR. pp. 538–547 (2017)

62. Reed, J.: Costa Rica State of Emergency Declared After Ransomware Attacks. <https://securityintelligence.com/news/costa-rica-state-emergency-ransomware/>
63. RSA: RSA SecurID Hardware Authenticators. <https://www.tokenguard.com/RSA-SecurID-Hardware.asp/> (2023)
64. Schnorr, C.P.: Efficient Identification and Signatures for Smart Cards. In: Proc. ASIACRYPT. pp. 239–252 (1989)
65. Shamir, A.: How to Share a Secret. Commun. ACM **22**(11), 612–613 (1979)
66. Shannon, C.E.: A Mathematical Theory of Communication. The Bell System Technical Journal **27**(3), 379–423 (1948)
67. Shoup, V.: A Computational Introduction to Number Theory and Algebra. Cambridge University Press, <https://shoup.net/ntb/ntb-v2.pdf>
68. Simon, T., Joo, H., Matthews, I., Sheikh, Y.: Hand Keypoint Detection in Single Images Using Multiview Bootstrapping. In: Proc. IEEE CVPR. pp. 1145–1153 (2017)
69. Srinivas, S., Balfanz, D., Tiffany, E., Czeskis, A.: Universal 2nd Factor (U2F) Overview. FIDO Alliance (2017)
70. Takahashi, A., Koda, Y., Ito, K., Aoki, T.: Fingerprint Feature Extraction by Combining Texture, Minutiae, and Frequency Spectrum Using Multi-Task CNN. In: Proc. IEEE IJCB. pp. 1–8 (2020)
71. Wang, D., Gu, Q., Huang, X., Wang, P.: Understanding Human-Chosen PINs: Characteristics, Distribution and Security. In: Proc. ACM ASIA CCS. pp. 372–385 (2017)
72. Wang, D., Wang, P., He, D., Tian, Y.: Birthday, Name and Bifacial-Security: Understanding Passwords of Chinese Web Users. In: Proc. USENIX Security. pp. 1537–1555 (2019)
73. Wang, D., Zhang, Z., Wang, P., Yan, J., Huang, X.: Targeted Online Password Guessing: An Underestimated Threat. In: Proc. ACM CCS. pp. 1242–1254 (2016)
74. Wang, M., Deng, W.: Mitigating Bias in Face Recognition Using Skewness-Aware Reinforcement Learning. In: Proc. IEEE CVPR. pp. 9322–9331 (2020)
75. Wu, C., He, K., Chen, J., Zhao, Z., Du, R.: Liveness Is not Enough: Enhancing Fingerprint Authentication with Behavioral Biometrics to Defeat Puppet Attacks. In: Proc. USENIX Security. pp. 2219–2236 (2020)
76. Wu, J., Ji, X., Lyu, Y., Luo, X., Meng, Y., Morales, E., Wang, D., Luo, X.: Touchscreens Can Reveal User Identity: Capacitive Plethysmogram-Based Biometrics. IEEE Trans. Mob. Comput. **23**(1), 895–908 (2024)
77. Yubikey: <https://www.yubico.com/products/yubikey-5-overview/>
78. Zhang, Y., Xu, C., Cheng, N., Shen, X.S.: Secure Password-Protected Encryption Key for Deduplicated Cloud Storage Systems. IEEE Trans. Dependable Secure Comput. **19**(4), 2789–2806 (2021)
79. Zhang, Y., Xu, C., Li, H., Yang, K., Cheng, N., Shen, X.: PROTECT: Efficient Password-Based Threshold Single-Sign-on Authentication for Mobile Users against Perpetual Leakage. IEEE Trans. Mob. Comput. **20**(6), 2297–2312 (2020)